

工科数分数学实验报告

院（系） 计算机系 实验时间： 2024.12.23
学号 JS124523 姓名 邱子豪

实验一

一、实验题目：设数列 $\{x_n\}$ 由下列递推关系式给出： $x_1 = \frac{1}{2}, x_{n+1} = x_n^2 + x_n (n = 1, 2, \dots)$,

观察数列 $\frac{1}{x_1 + 1} + \frac{1}{x_2 + 1} + \dots + \frac{1}{x_n + 1}$ 的极限。

二、实验目的和意义

- 1:掌握 mathematica 数学实验的基本用法。
- 2: 学会利用 mathematica 编程求数列极限。
- 3: 了解函数与数列的关系。

三、程序设计

```
f[x_] := x^2 + x; xn = 0.5; g[x_, y_] := y + 1 / (1 + x); yn = 0;
For[n = 1, n <= 15, n++, xN = xn; yN = yn; xn = N[f[xN]];
  [yn = N[g[xN, yN]]]; Print[yn];]
Print["y15=", yn]
```

四、程序运行结果

```
0.666667
1.2381
1.67053
1.91835
1.99384
1.99996
2.
2.
2.
2.
2.
2.
2.
2.
2.
y15=2.
```

五、结果的讨论和分析

这个实验中，当 y_n 中 n 趋向无穷大的时候，能够更加接近极限，当取 15 以上的时候，数据

稳定在 2 左右，那么 2 就是极限值。

实验二

一、实验题目：已知函数 $f(x) = \frac{1}{x^2 + 2x + c}$ ($-5 \leq x \leq 4$)，作出并比较当 c 分别取 -1, 0, 1, 2, 3 时的图形，并从图上观察极值点、驻点、单调区间、凹凸区间以及渐近线。

二、实验目的和意义：

1. 通过实验掌握如何用 mathematica 作图。

2. 学会观察图像来求函数的相关数据。

三、程序设计

```
f[x_] := 1 / (x^2 + 2 * x + (-1))
Plot[f[x], {x, -5, 4}, GridLines -> Automatic, Frame -> True,
  PlotStyle -> RGBColor[1, 0, 0]]
```

```
f[x_] := 1 / (x^2 + 2 * x + 0)
Plot[f[x], {x, -5, 4}, GridLines -> Automatic, Frame -> True,
  PlotStyle -> RGBColor[1, 0, 0]]
```

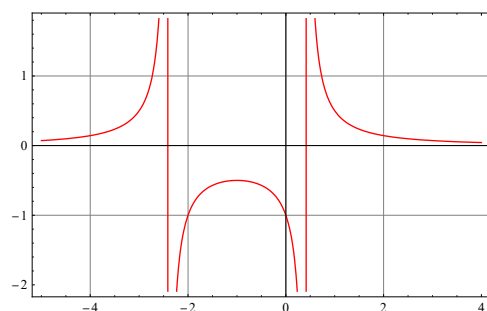
```
f[x_] := 1 / (x^2 + 2 * x + 1)
Plot[f[x], {x, -5, 4}, GridLines -> Automatic, Frame -> True,
  PlotStyle -> RGBColor[1, 0, 0]]
```

```
f[x_] := 1 / (x^2 + 2 * x + 2)
Plot[f[x], {x, -5, 4}, GridLines -> Automatic, Frame -> True,
  PlotStyle -> RGBColor[1, 0, 0]]
```

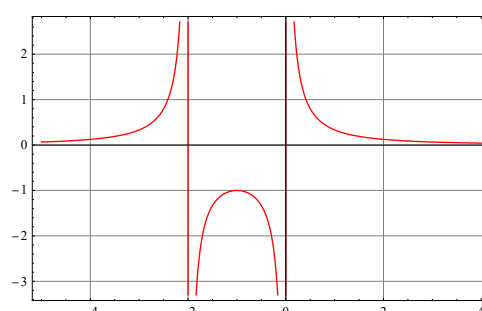
```
f[x_] := 1 / (x^2 + 2 * x + 3)
Plot[f[x], {x, -5, 4}, GridLines -> Automatic, Frame -> True,
  PlotStyle -> RGBColor[1, 0, 0]]
```

四、程序运行结果

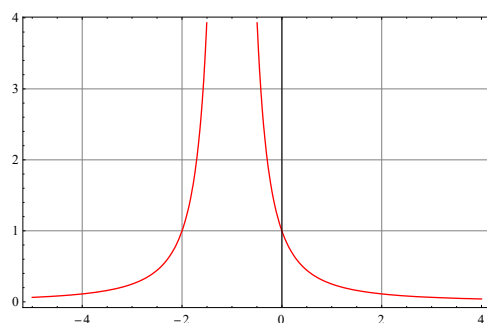
C=-1



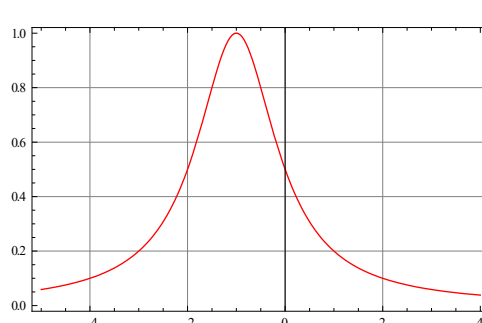
C=0



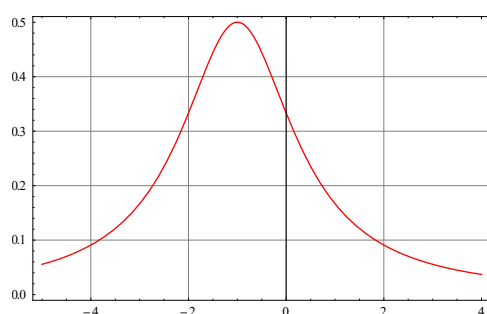
C=1



C=2



C=3



五、结果的讨论和分析

c 不同时，函数的形态有较大的不同，也就是原方程=0 什么情况下有解的问题，根据图像很容易的得到驻点，拐点等相关信息。

实验三

实验题目：作出函数 $y = \ln(\cos x^2 + \sin x)$ ($-\frac{\pi}{4} \leq x \leq \frac{\pi}{4}$) 的函数图形和泰勒展开式（选

取不同的 x_0 和 n 值）图形，并将图形进行比较。

二、实验目的和意义：利用Mathematica计算函数 $f(x)$ 的各阶泰勒多项式，并通过绘制曲线图形，进一步掌握泰勒展开与函数逼近的思想。

三、计算公式一个函数 $f(x)$ 若在点 x_0 的邻域内足够光滑，则在该邻域内有泰勒公式

$$f(x) = f(x_0) + \sum_{k=1}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k + o(|x - x_0|^n) \text{ 当 } |x - x_0| \text{ 很小时, 有}$$

$$f(x) \approx f(x_0) + \sum_{k=1}^n \frac{f^{(k)}(x_0)}{k!} (x - x_0)^k$$

四、程序设计

```
f[x_] := Log[Cos[x^2] + Sin[x]]

xmin = -Pi / 4;
xmax = Pi / 4;

Plot[f[x], {x, xmin, xmax}]

TaylorSeries[x0_, n_] :=
Module[{series}, series = Normal[Series[f[x], {x, x0, n}]];
Return[series]]

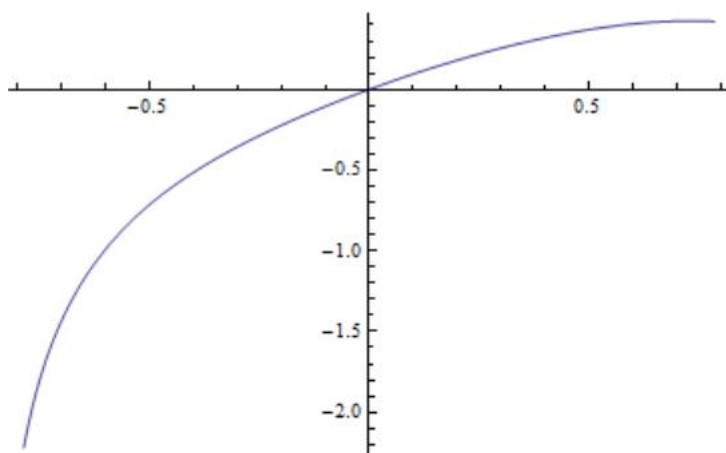
ComparisonPlot[x0_, nterms_] :=
Module[{taylor}, taylor = TaylorSeries[x0, nterms];
Plot[{f[x], taylor}, {x, xmin, xmax}, PlotStyle -> {Blue, Red},
PlotLegends -> {"Original Function",
StringForm["Taylor Series at x0=`1` (n=`2`)", x0, nterms]},
AxesLabel -> {"x", "y"},
PlotLabel -> StringForm["Taylor Series Expansion at x0=`1`", x0],
GridLines -> Automatic]]

x0Values = {0, Pi / 8, -Pi / 8};
Grid[Table[Column[{ComparisonPlot[x0, 4], ComparisonPlot[x0, 7]}],
{x0, x0Values}]]

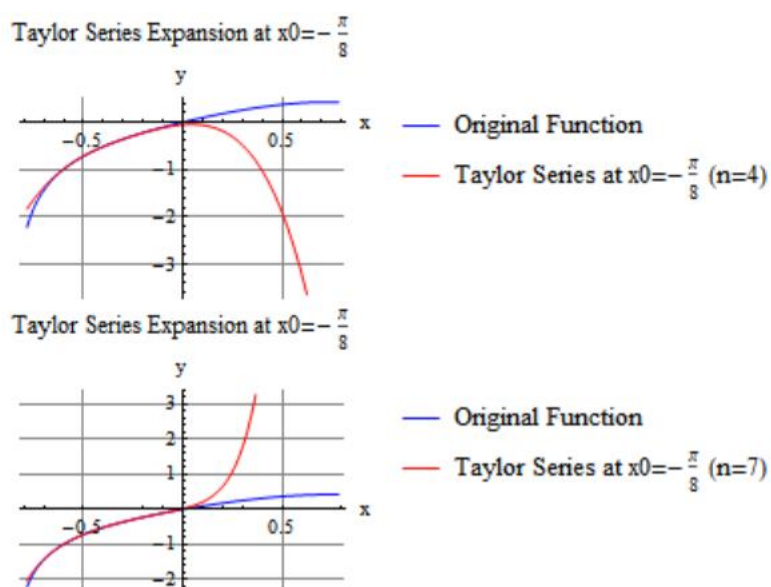
TaylorCoefficients = Table[D[f[x], {x, n}] /. x -> 0, {n, 0, 7}];
Print["Taylor coefficients at x0=0:"];
TableForm[Table[{n, TaylorCoefficients[[n+1]]}, {n, 0, 7}],
TableHeadings -> {None, {"Order", "Coefficient"}}]
```

五、程序运行结果

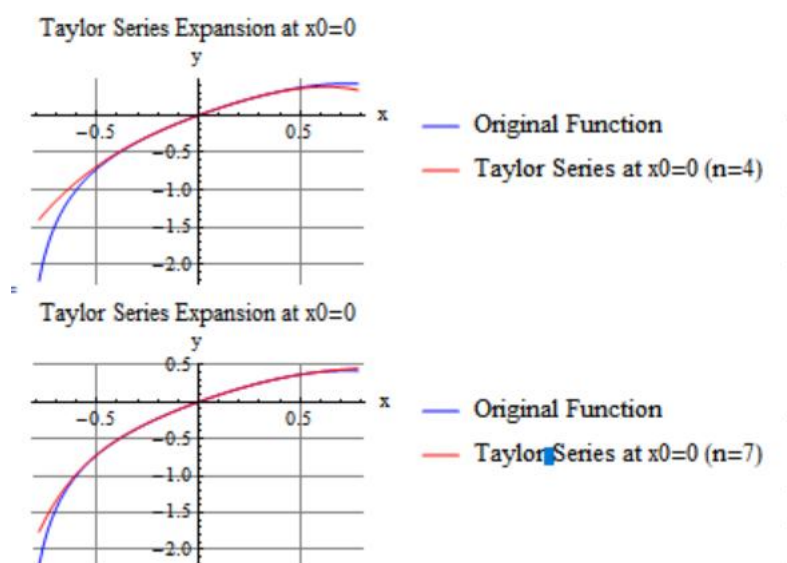
$F[x]$ 的图像



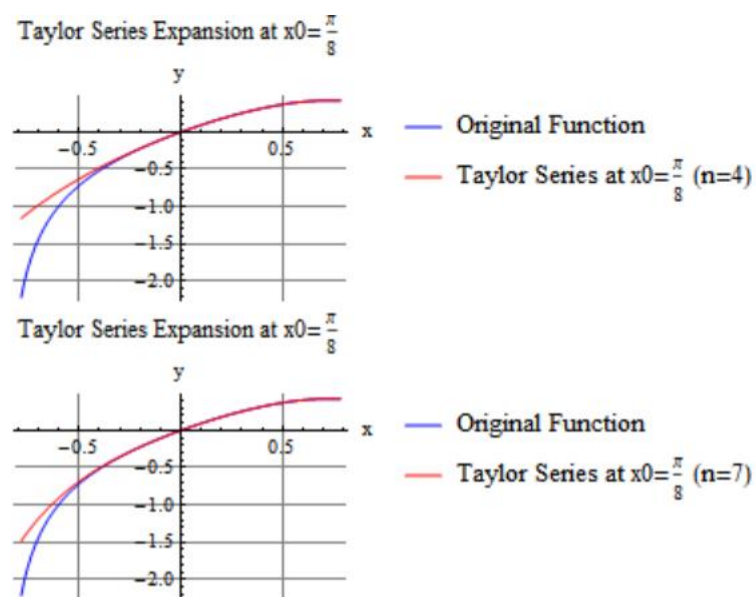
$X = -\pi/8$, $n=4, 7$ 时的泰勒展开图像



$X=0$, $n=4, 7$ 时的泰勒展开图像



$X=\pi/8$, $n=4, 7$ 时的泰勒展开图像



六、结果的讨论和分析： 随着 n 值的增大，泰勒公式的函数越来越趋向于原函数

实验四

一、实验题目 分别用梯形法、抛物线法计算定积分 $\int_0^{\frac{\pi}{2}} \sin x^2 dx$ 的近似值（精确到 0.0001）。

二、实验目的和意义：通过实验，可以更深入地理解梯形法和抛物线法（辛普森法）这两种常用的数值积分方法。将数值分析的理论知识与实际计算相结合，更好地理解 and 掌握数值积分的概念。

三、程序设计

```
f[x_] := Sin[x^2]

a = 0;
b = Pi/2;
m2 = N[D[D[f[x], x], x] /. x -> 0];
m4 = N[D[D[D[D[f[x], x], x], x], x] /. x -> 0]; delta = 10^-4; n0 = 40;
t[n_] := (b - a) / n * ((f[a] + f[b]) / 2 + Sum[f[a + i * (b - a) / n], {i, 1, n - 1}])

SimpsonsRule[a_, b_, n_] := Module[{h, xValues, sum1, sum2}, h = (b - a) / n;
  xValues = Table[a + i * h, {i, 0, n}];
  sum1 = Sum[f[xValues[[2 * i]]], {i, 1, n / 2}];
  sum2 = Sum[f[xValues[[2 * i - 1]]], {i, 1, n / 2}];
  (h / 3) * (f[First[xValues]] + f[Last[xValues]] + 4 * sum2 + 2 * sum1)]

Print["梯形法计算结果: "];
Print["-----"];
n = 1;
While[True, result = N[t[n]];
  Print[NumberForm[N[n], {8, 6}], " ", NumberForm[result, {8, 6}]];
  error = Abs[(b - a)^3 / (12 * n^2) * m2];
  If[error < delta, Break[]];
  If[n = n0, Break[]];
  n = n + 1;]

a = 0.0;
b = 2.0;
Print["\n抛物线法计算结果: "];
Print["-----"];
simpResults =
  Table[{NumberForm[N[n], {8, 6}],
    NumberForm[SimpsonsRule[a, b, 2 * n], {8, 6}]}, {n, 1, 15}];
Print[TableForm[simpResults, TableAlignments -> Right]];

exactValue = NumberForm[NIntegrate[f[x], {x, 0, Pi / 2}], {8, 6}];
Print["\n积分精确值: ", exactValue]
```

四、程序运行结果

梯形法计算结果：

```
-----  
1.000000  0.490297  
2.000000  0.699477  
3.000000  0.771019  
4.000000  0.796208  
5.000000  0.807773  
6.000000  0.814021  
7.000000  0.817775  
8.000000  0.820206  
9.000000  0.821871  
10.000000 0.823060  
11.000000 0.823939  
12.000000 0.824607  
13.000000 0.825127  
14.000000 0.825539  
15.000000 0.825871  
16.000000 0.826144  
17.000000 0.826369  
18.000000 0.826558  
19.000000 0.826718  
20.000000 0.826854  
21.000000 0.826971  
22.000000 0.827073  
23.000000 0.827162  
24.000000 0.827240  
25.000000 0.827309  
26.000000 0.827370  
27.000000 0.827424  
28.000000 0.827472  
29.000000 0.827516  
30.000000 0.827555  
31.000000 0.827591  
32.000000 0.827623  
33.000000 0.827653  
34.000000 0.827680
```


35.000000	0.827704
36.000000	0.827727
37.000000	0.827748
38.000000	0.827767
39.000000	0.827785
40.000000	0.827801

抛物线法计算结果：

1.000000	0.308713
2.000000	0.776673
3.000000	0.832671
4.000000	0.838372
5.000000	0.836927
6.000000	0.834307
7.000000	0.831704
8.000000	0.829375
9.000000	0.827348
10.000000	0.825595
11.000000	0.824074
12.000000	0.822748
13.000000	0.821585
14.000000	0.820559
15.000000	0.819648

积分精确值：0.828116

五、结果的讨论和分析：梯形法在计算上相对简单，但可能需要更多的迭代次数来达到相同的精度。而抛物线法虽然计算复杂度稍高，但通常能在较少的迭代次数内达到所需的精度，显示出更高的计算效率。